1. Executive Summary

Author: Sam Bederman

GitHub Fork

GitHub - b3at1/csce413_assignment2

YouTube Video

PART 1: Reconnaissance
PART 2: MITM
PART 3: Security Fixes

Overview

In this assignment, I was engaged by CSCE-413 to develop network tooling, perform comprehensive reconnaissance, and conduct a network-based vulnerability assessment. My objectives included creating specialized tools to enhance the organization's security posture and providing strategic remediation for any identified weaknesses. The engagement was successful in uncovering numerous security flaws, which ultimately led to the discovery and capture of three specific flags during the exercise.

Critical Vulnerabilities Found

The assessment identified several high-impact vulnerabilities that could allow an adversary to compromise the environment. The primary MITRE ATT&CK techniques observed include:

T1557 – Adversary-in-the-Middle (AiTM): Exploited to intercept sensitive traffic and credentials.

T1021.004 – Remote Services (SSH): Leveraged through weak authentication to gain initial access.

T1550.001 – Use Alternate Authentication Material (Application Access Token): Used to impersonate authorized users and bypass standard auth flows via stolen API tokens.

Recommended Fixes

A detailed analysis of the recommended security controls will be provided in Part 3 of this report. At a high level, the proposed remediation strategy focuses on implementing robust identity management through Multi-Factor Authentication (MFA) and transitioning to SSH key-based authentication to replace vulnerable password schemas. Additionally, the security posture should be reinforced by implementing strict network segmentation and enforcing securely encrypted network protocols (TLS 1.2+) to protect data in transit and prevent credential sniffing.

2. Part 1: Reconnaissance

Port Scanner Implementation

The port scanner implementation consistend of the minimum requirements:

- Accept target IP/hostname and port range as arguments ✅
- Perform TCP connect scans to detect open ports ✅
- Display results showing port number, state (open/closed), and timing ✅
- Handle errors gracefully (timeouts, connection refused, etc.) ✅
- Service/banner detection - Identify what service is running on each port ✅ (only if the service responds with a banner)

  I also implemented the following extras:
- Scan multiple hosts (CIDR notation) ✅
- Different verbosity / display levels ✅
- Output formats (supports html, csv, txt, json) ✅
  The scanner works as a 4 step process. First `main()` parses cli args, such as using the `ipaddress` library to convert a CIDR range into a range of targets. `scan_range` then loops over every IP, and for each IP loops over every port number. Then, inside the loop `scan_port` is called which does the actual scanning by first attempting a TCP handshake. If successful it attempts to identify the service by trying to receive the banner and sending more requests such as a HEAD request to identify the type of service. Lastly the returned tuples are incorporated into a list that is fed into `display_results` to output the scan results in the correct format.

Discovered Services

The discovered services were as follows:

Found 7 open ports

- Target: 172.20.0.1 | Port 1716: open (scanned in 0.9512 seconds) | Service: null
- Target: 172.20.0.1 | Port 5001: open (scanned in 0.5029 seconds) | Service: http
- Target: 172.20.0.10 | Port 5000: open (scanned in 0.5029 seconds) | Service: http
- Target: 172.20.0.11 | Port 3306: open (scanned in 0.0002 seconds) | Service: mysql
- Target: 172.20.0.20 | Port 2222: open (scanned in 0.0105 seconds) | Service: ssh
- Target: 172.20.0.21 | Port 8888: open (scanned in 0.5704 seconds) | Service: http
- Target: 172.20.0.22 | Port 6379: open (scanned in 0.5007 seconds) | Service: null

Methodology

The methodology was scanning the 172.20.0.0/24 subnet from ports 1-9000. The scan results were used to identify services of interest to exploit such as 172.20.0.10:5000 which was exploited with the MITM attack to get `FLAG{n3tw0rk_tr4ff1c_1s_n0t_s3cur3}` as well as 172.20.0.21:8888 which utilized the first flag to get flag 3 `FLAG{p0rt_kn0ck1ng_4nd_h0n3yp0ts_s4v3_th3_d4y}` and 172.20.0.20:2222 which was exploited for flag 2 `FLAG{h1dd3n_s3rv1c3s_n33d_pr0t3ct10n}`

3. Part 2: MITM Attack

Vulnerability Analysis

The primary vulnerability identified is a lack of encryption for sensitive data in transit, facilitating an MITM attack. By intercepting unencrypted HTTP traffic, an attacker can capture the API authentication token (Flag 1).

The second vulnerability involves a Remote SSH service configured with weak or default credentials. This allows an adversary to perform a brute-force or credential-guessing attack to gain shell access to the host.

The third vulnerability is an insecure API endpoint that trusts stolen api access tokens (Flag 1). An attacker can use this "alternate authentication material" to impersonate a legitimate user and submit malicious requests to extract further sensitive data (Flag 3).

Attack Methodology

To execute the attack, I positioned myself on the local network and used Wireshark to perform packet sniffing on the target interface. By monitoring the communication between the client and the server at 172.20.0.10, I was able to intercept the raw, unencrypted HTTP GET request and the subsequent server response.

Captured Data

```
GET /api/secrets HTTP/1.1
Host: 172.20.0.10:5000
Connection: keep-alive
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/140.0.0.0 Safari/537.36
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp
,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Referer: [http://172.20.0.10:5000/](http://172.20.0.10:5000/)
Accept-Encoding: gzip, deflate
Accept-Language: en-US,en;q=0.9


HTTP/1.1 200 OK
Server: Werkzeug/3.1.5 Python/3.11.14
Date: Sat, 31 Jan 2026 19:05:34 GMT
Content-Type: application/json
Content-Length: 214
Connection: close

[
  {
    "description": "API authentication token for secret services - MITM
attack will reveal this!",
```

```
    "id": 1,
    "secret_name": "api_token",
    "secret_value": "FLAG{n3tw0rk_tr4ff1c_1s_n0t_s3cur3}"
  }
]
```


Real-World Impact Assessment

The impact of an MITM attack is severe, as it grants an attacker complete visibility into "plaintext" credentials and session tokens. Once an API token is stolen, an attacker can bypass standard authentication barriers to perform unauthorized actions, such as data exfiltration or system configuration changes. This scenario demonstrates "vulnerability chaining" where a simple lack of encryption leads to identity theft (SSH/API), potentially resulting in elevated privileges within the infrastructure, or even full system compromise.

4. Part 3: Security Fixes

Port Knocking

Design Decisions

The system utilizes a split architecture where `knock_server.py` runs within the SSH container, while the port knocking container hosts the client and `demo.py`. Additional features include a timeout mechanism to reset the sequence window and a strict reset policy that clears progress upon receiving an incorrect knock.

Implementation Details

The core logic resides in `knock_server.py`, where `listen_for_knocks` initializes non-blocking sockets for each port in the sequence. The main event loop uses `select.select` to handle incoming connections. For each connection, the server checks the `ip_states` dictionary to validate the sequence. If the port matches the expected value, the state advances; if it is incorrect or if the `window_seconds` timeout is exceeded, the state resets to zero. When the full sequence is verified, `change_protected_port` is called to execute the necessary `iptables` command, inserting a rule to accept traffic on the protected port.

The demo is handled by `demo.py` which coordinates the workflow by first calling `attempt_ssh` to verify the port is closed. It then spawns the `knock_client.py` process, which triggers `perform_knock_sequence` to iterate through the required ports using `send_knock` for each TCP handshake. Finally, `demo.py` calls `attempt_ssh` again to confirm that the firewall rule has been applied and access is granted.

Security Analysis

Port knocking restricts access by keeping the SSH port closed until a specific sequence of connection attempts is detected. This obscures the service from casual scanners and automated attacks, adding a layer of defense-in-depth dependent on the secrecy of the knock sequence.

Limitations and Improvements

Currently, the knocking ports reject connections rather than dropping packets, making them visible to scanners and potentially reducing the search space for an attacker. Security could be improved by configuring the firewall to silently drop packets, increasing the sequence complexity, and introducing dummy ports with randomized behavior to obfuscate the true knock sequence.

Honeypot

Architecture and Design

I chose to implement a honeypot that emulates the secret_api service. It runs on the same port (8888) and has the same exact functions, but all user interactions are logged in and the sensitive api outputs are replaced with dummy data that attempts to convince an attacker that it is legitimate. The honeypot works exactly the same as `api.py` but has the added feature of calling `setup_logging` in logger.py which is then utilized to intercept and log every single request made to the honeypot.

Logging Mechanisms & Capabilities

All requests have the timestamp, ip address, port, request type, and url logged. Additionally if the request contains a token (api_token) that gets logged as well. The idea is that if a legitimate token from the real API is used on the honeypot api, we know that the token has been compromised and can automatically revoke it so it cannot be abused in our real systems.

Analysis of Captured Attack

Here is an example of a captured attack:

```
{"timestamp": "2026-01-31T23:00:00.816470", "ip_address": "172.20.0.1",
"port": 55900, "http_request_type": "GET", "url":
"http://172.20.0.30:8888/flag", "api_token": null}
172.20.0.1 - - [31/Jan/2026 23:00:00] "□[31m□[1mGET /flag HTTP/1.1□[0m" 401
-
{"timestamp": "2026-01-31T23:00:02.634680", "ip_address": "172.20.0.1",
"port": 55916, "http_request_type": "GET", "url":
"http://172.20.0.30:8888/", "api_token": null}
172.20.0.1 - - [31/Jan/2026 23:00:02] "GET / HTTP/1.1" 200 -
{"timestamp": "2026-01-31T23:00:08.357873", "ip_address": "172.20.0.1",
"port": 57374, "http_request_type": "GET", "url":
"http://172.20.0.30:8888/data", "api_token": null}
172.20.0.1 - - [31/Jan/2026 23:00:08] "□[31m□[1mGET /data HTTP/1.1□[0m" 401
-
{"timestamp": "2026-01-31T23:00:15.088240", "ip_address": "172.20.0.1",
"port": 57386, "http_request_type": "GET", "url":
"http://172.20.0.30:8888/data?token=FLAG{n3tw0rk_tr4ff1c_1s_n0t_s3cur3}",
"api_token": "FLAG{n3tw0rk_tr4ff1c_1s_n0t_s3cur3}"}
172.20.0.1 - - [31/Jan/2026 23:00:15] "GET /data?
token=FLAG{n3tw0rk_tr4ff1c_1s_n0t_s3cur3} HTTP/1.1" 200 -
```

As you can see in these logs, a simple IP attempted to access `/flag` without a token before making a few GET requests to the API directly. It then successfully accesses `/data` with a valid token. From this attack we can gleam that the attacker IP is an unwanted system since they're interacting with the honeypot. We also know that the api_token they used is compromised since legitimate users known not to interact with the honeypot.

5. Remediation Recommendations

How to fix the MITM vulnerability (TLS/SSL)

The most effective remediation for the Man-in-the-Middle (MITM) vulnerability is enforcing HTTPS with strong TLS encryption (1.2+). By provisioning valid SSL/TLS certificates and disabling unencrypted HTTP traffic, sensitive data like API tokens is protected in transit. This prevents attackers from sniffing credentials from the wire, directly mitigating the "Use Alternate Authentication Material" vulnerability.

Best Practices for Hidden Services

Security by obscurity is insufficient on its own. While port knocking obfuscates the SSH service, it is merely a defense-in-depth measure. Detailed scanning or timing analysis can still reveal hidden ports. Therefore, the primary focus must be on hardening the service itself by enforcing public key-based authentication and disabling password logins to prevent brute-force attacks.

Network Segmentation Strategies

Network segmentation limits the blast radius of a breach by separating services into distinct subnets. Isolating the public-facing web app in a DMZ while keeping databases and internal APIs in a restricted backend subnet makes lateral movement significantly harder. Strict "allow-list" firewall rules should enforce this separation, permitting only essential inter-service traffic.

Monitoring and Detection Recommendations

While the honeypot serves as a targeted alert system, comprehensive monitoring is required for the entire infrastructure. Host-based intrusion detection systems (HIDS) or agents like osquery and Sysmon should be deployed to capture process execution and network connections. Aggregating these logs into a SIEM allows for real-time anomaly detection and correlation of events across the environment.

6. Conclusion

Lessons Learned

The engagement provided valuable hands-on experience in both offensive and defensive network security operations. The primary lessons and skills acquired include:

- Tool Development: designing and implementing a custom threaded port scanner to identify open services across a subnet.
- Reconnaissance: Mapping out attack surfaces and entry points using the custom network scanner.
- Vulnerability Assessment: identifying critical network-based vulnerabilities, such as unencrypted traffic (MITM) and weak authentication mechanisms.
- Exploit Chaining: demonstrating how seemingly minor vulnerabilities can be chained together (e.x. MITM --> Stolen Token --> Sensitive Data Access) to achieve system compromise.
- Security via Obscurity: implementing security controls like port knocking to obscure critical services from automated scanners.
- Active Defense: deploying honeypots to detect active threats and gathering threat intelligence on attacker behavior.

Future Work

To further enhance the security posture of the network, the following improvements are recommended for future iterations:

- Network Segmentation:
    - Implement VLANs to isolate critical infrastructure from user workstations.
    - Enforce micro-segmentation policies to restrict traffic between containers.
- SIEM & Monitoring:
    - Centralize logs from all services into a SIEM (e.g., ELK Stack, Splunk) for unified visibility.
    - Deploy Network Intrusion Detection/Prevention Systems (NIDS/NIPS) like Snort or Suricata.
- Honeypot Augmentation:
    - Expand the honeypot infrastructure to emulate other high-value targets (e.x. database, ssh honeypots).
    - Integrate automated alerting pipelines (email or slack) to notify SOC teams of breaches.
- Authentication Hardening:
    - Implement Multi-Factor Authentication (MFA) for all remote access services.
    - Rotate API keys and secrets regularly using a secrets management vault.

Attributions

Agentic AI (Gemini 3 Pro) was utilized in Visual Studio Code to modify and restructure scripts as well as to debug. Additionally, the following external chats were utilized:
https://gemini.google.com/share/8dcbdf405d14
https://gemini.google.com/share/e0d2f0bb0b89